



Sincronização em TypeScript: Jantar dos Filósofos

Atividade 2.1 - Sistemas Operacionais (Prof. L. A. Minora)

Equipe: **Antônio Irineu**, **Maycon Alexsander** e **Lucas Vinicius**

TS

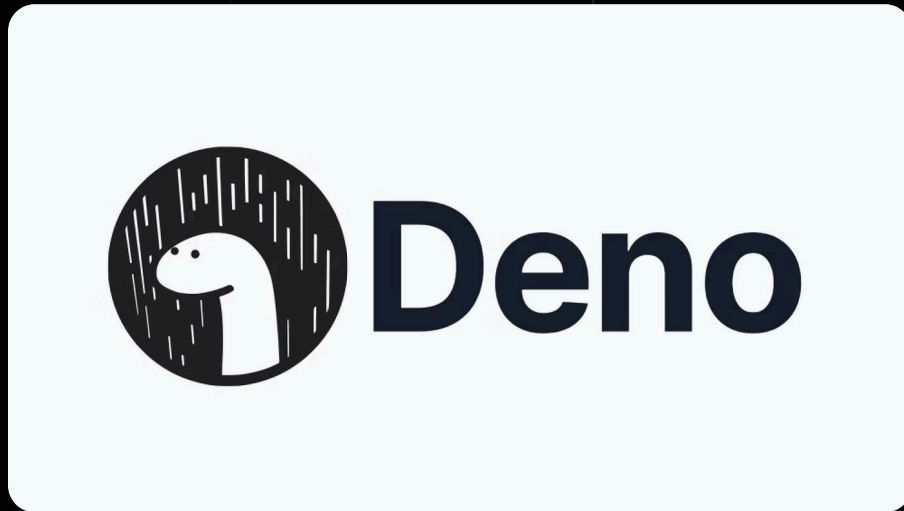
Made with GAMMA

Sumário

1. Linguagem e Grupo
2. Sumário
3. Bibliotecas Usadas
4. Tratamento da Concorrência no Código
5. Dificuldades
6. Observações do Grupo

Bibliotecas Usadas

Deno



Foi utilizado o **Deno**, um runtime moderno para TypeScript.

async-mutex



Foi utilizada a biblioteca **async-mutex**, uma implementação de primitivas de sincronização baseada em Promises.

- **Recurso utilizado:** Classe Semaphore.
- Por **que foi necessária?**
 - O TypeScript executa em uma **única thread** (*Event Loop*).
 - Semáforos tradicionais bloqueiam a thread no Sistema Operacional, o que **congelaria** todo o programa em TypeScript.
 - A biblioteca **async-mutex** implementa Semáforos baseados em Promises, permitindo a exclusão mútua assíncrona sem travar a execução do Event Loop.

Tratamento da Concorrência

Implementação: Semáforos

```
// Semáforo limitador da mesa quebra a Espera Circular e previne Deadlocks
const mesa = new Semaphore(N - 1);

// Um semáforo de capacidade 1 (Mutex) para garantir exclusão mútua de cada garfo
const garfos = Array.from({ length: N }, () => new Semaphore(1));

// [...]

// Tentativa de acesso aos recursos compartilhados
const [, releaseMesa] = await mesa.acquire();

const [, releaseEsquerdo] = await garfos[esquerdo].acquire();
const [, releaseDireito] = await garfos[direito].acquire();

await comer(id);

// Liberação dos recursos
releaseDireito();
releaseEsquerdo();
releaseMesa();
```

Execução: Prevenção de Deadlock

[colocar gif do terminal rodando]

Dificuldades



Compreender a diferença prática entre *Threads reais do S.O.* (que podem ser suspensas fisicamente) e o modelo *Single-Thread* do TypeScript.



Adaptar a lógica de concorrência e bloqueios do Python para o TypeScript, garantindo que o `await` fosse usado corretamente para não congelar o Event Loop.

Demais Observações

O uso de um semáforo controlador de capacidade (mesa = N - 1) demonstrou na prática a quebra da condição de **Espera Circular**, o que preveniu o *Deadlock* com sucesso.

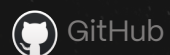
```
// Limita a N-1 filósofos tentando comer ao mesmo tempo  
const mesa = new Semaphore(N - 1);
```

As *Promises* do TypeScript mostraram-se uma alternativa viável para simular problemas clássicos de S.O. através de concorrência não-bloqueante.

```
// Aguarda a execução concorrente de todas as rotinas  
await Promise.all(tarefas);
```

Obrigado!

Dúvidas?



GitHub – mayconalex/2026-1-at...

Atividade avaliativa número 1 do 2o
bimestre de 2026.1 da disciplina de...